

Experiment No.	3
Experiment Title	Regularized Regression Models for Loan Sanction Amount Prediction ¹
Student Name	S.R.Aadhithya
Register Number	3122247001001
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

This experiment predicts a continuous target, Loan Sanction Amount (USD), from applicant demographic, financial, and property attributes. Four regression approaches - plain Linear Regression, Ridge, Lasso, and Elastic Net - are trained and compared, with Ridge, Lasso and Elastic Net tuned via grid search over their regularization strength. The point isn't just to find the best test score, but to see what regularization actually changes about the model, which the coefficient comparison later in this report makes fairly clear.

2 Problem Statement

Given a raw applicant record - gender, age, income, employment type, credit history, existing loan expenses, property details, and so on - the task is to predict the loan sanction amount that will be approved, in US dollars. The input is `train.csv`, 30,000 rows across 24 columns with a mix of numeric and categorical fields, several of them with missing values. The output is a single continuous value per applicant, so this is a regression problem rather than a classification one, and the model is judged on error magnitude (MAE, MSE, RMSE) and variance explained (R^2) rather than accuracy.

3 Dataset Description

Dataset Name	Loan Sanction Amount Prediction (<code>train.csv</code>)
Dataset Source	Provided course dataset
Number of Samples	30,000 raw rows; 29,660 after dropping rows with a missing target
Number of Features	24 raw columns → 21 predictors after dropping Customer ID and Name ; top 10 retained after feature selection
Number of Classes	N/A - regression target (Loan Sanction Amount, USD)
Missing Values	Present in 10 of 24 columns; worst affected are Type of Employment (~24%), Property Age (~16%), and Income (~15%). 340 rows (1.1%) had a missing target and were dropped outright.
Train-Test Split	70% train / 15% validation / 15% test (20,761 / 4,450 / 4,449 rows)

4 Exploratory Data Analysis

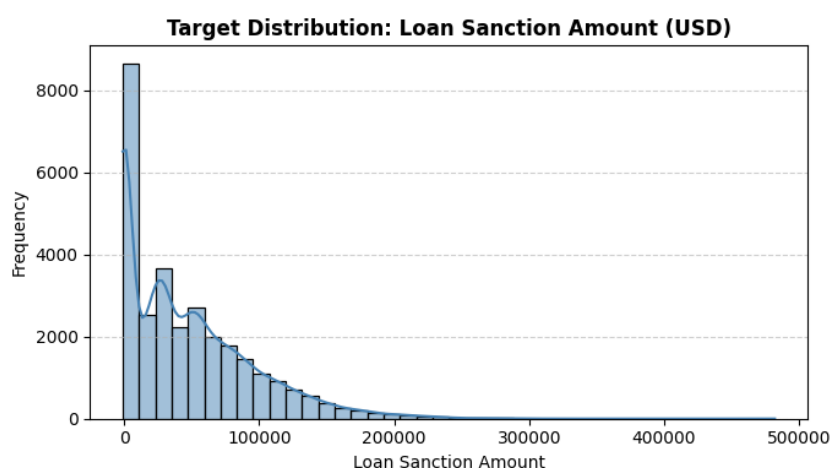


Figure 1: Distribution of the target - Loan Sanction Amount (USD)

Inference: The target spans a wide range of loan amounts, as you'd expect from a population with very different income and property values. A KDE curve is overlaid on the histogram, and the spread here is really the

whole story of why this is a hard regression problem - predicting a dollar figure from indirect signals like income bracket and credit score leaves a lot of room for error, which shows up later in the fairly modest R^2 scores (all four models land around 0.57).

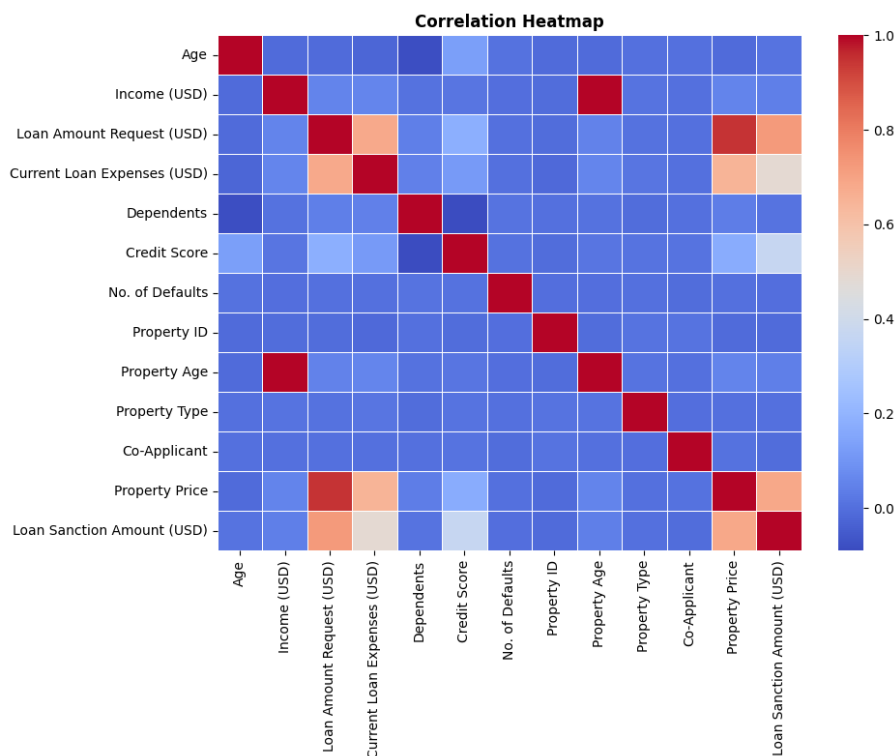


Figure 2: Correlation heatmap of numeric features

Inference: One thing worth flagging here rather than glossing over: looking back at the raw data preview, the **Income (USD)** and **Property Age** columns contain identical values row for row (e.g. 1933.05, 4952.91, 988.19 - the same numbers appear in both columns for the same applicants). That’s almost certainly a data issue rather than a real relationship, and it shows up later as instability in the plain Linear Regression coefficients. Beyond that, the financial columns (loan amount, property price, income) show the kind of mutual correlation you’d expect, since larger loans tend to go with larger incomes and more expensive properties.

5 Feature Selection

`SelectKBest` with `f_regression` (the ANOVA F-test adapted for a continuous target) was used to rank all 21 predictors and keep the top 10:

- Income (USD)

- Income Stability
- Profession
- Location
- Loan Amount Request (USD)
- Current Loan Expenses (USD)
- Expense Type 1
- Credit Score
- Property Age
- Property Price

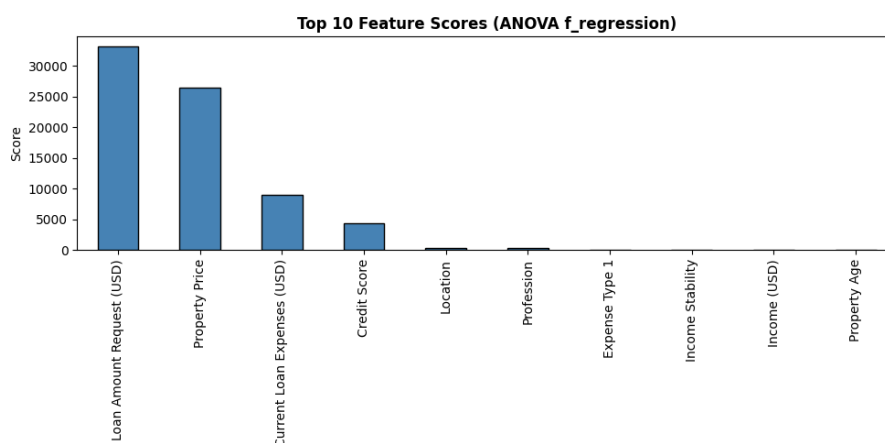


Figure 3: Top 10 feature scores (ANOVA `f_regression`)

Inference: The scores drop off noticeably after the first couple of features, which is fairly typical of this kind of ranking - a small handful of variables carry most of the linearly-detectable signal, and the rest contribute smaller, incremental amounts. Given the Income/Property Age duplication noted above, it's worth treating the very top of this ranking with a little healthy suspicion rather than at face value.

6 Data Preprocessing

Rows with a missing target (340 of them) were dropped first, since there's nothing to learn from a record with no label. `Customer ID` and `Name` were

dropped as identifier columns with no predictive value. Remaining missing values were filled column-by-column - median for numeric columns, mode for categorical ones - rather than dropping any more rows, since several columns (Type of Employment in particular, at roughly 24% missing) would have cost a lot of data if those rows were removed instead. All categorical columns were then label-encoded, and the full feature set was standardized with `StandardScaler` before feature selection and modelling. No additional feature engineering was performed beyond this. One caveat worth noting: label encoding assigns an arbitrary numeric order to columns like `Profession` and `Location` that don't have a natural order, which regression models will treat as though it were meaningful - a one-hot encoding would avoid that, at the cost of more columns.

7 Machine Learning Algorithm

All four models here are linear - they differ only in how (and whether) they penalize large coefficients.

Linear Regression (OLS) fits coefficients β that minimize the residual sum of squares:

$$\hat{\beta} = \arg \min_{\beta} \sum_i (y_i - X_i \beta)^2$$

It's simple and interpretable, but has no protection against multicollinearity - when predictors are correlated (or, worse, effectively duplicated, as with Income and Property Age here), coefficients can swing to extreme, unstable values, which is exactly what happens with the Income coefficient in the results below.

Ridge Regression adds an L2 penalty on coefficient size:

$$\hat{\beta} = \arg \min_{\beta} \sum_i (y_i - X_i \beta)^2 + \alpha \sum_j \beta_j^2$$

This shrinks all coefficients toward zero (more aggressively for larger α) without forcing any of them to exactly zero, which makes it a natural fix for the multicollinearity problem OLS runs into.

Lasso Regression uses an L1 penalty instead:

$$\hat{\beta} = \arg \min_{\beta} \sum_i (y_i - X_i \beta)^2 + \alpha \sum_j |\beta_j|$$

The L1 penalty can drive coefficients to exactly zero, effectively performing feature selection as part of training - visible directly in the results table, where Lasso zeroes out the Income coefficient entirely.

Elastic Net blends both penalties, controlled by `l1_ratio` (1.0 = pure Lasso, 0.0 = pure Ridge):

$$\hat{\beta} = \arg \min_{\beta} \sum_i (y_i - X_i \beta)^2 + \alpha \left(r \sum_j |\beta_j| + \frac{1-r}{2} \sum_j \beta_j^2 \right)$$

This is useful when you want some sparsity (like Lasso) but also want correlated features to shrink together rather than have one arbitrarily zeroed out (a known weakness of pure Lasso).

Advantages (regularized models over plain OLS): more stable coefficients under multicollinearity, reduced overfitting, and (for Lasso/Elastic Net) built-in feature selection.

Limitations: an extra hyperparameter (α , and for Elastic Net, `l1_ratio`) has to be tuned; all four models remain fundamentally linear, so none of them can capture non-linear relationships between features and the target even with penalties applied.

8 Experimental Setup

Python Version

Scikit-Learn Version

IDE

Operating System

Hardware

Environment Activation `source /opt/anaconda3/bin/activate anaconda-navig`

9 Hyperparameter Selection

Grid search (5-fold cross-validation, scored on R^2) was used to tune Ridge, Lasso, and Elastic Net on the training set.

Parameter	Value / Grid Searched
Ridge: alpha grid	{0.1, 1.0, 10.0, 100.0} - best: 100.0
Lasso: alpha grid	{0.1, 1.0, 10.0, 50.0} - best: 50.0
Lasso / Elastic Net: max_iter	5000
Elastic Net: alpha grid	{0.1, 1.0} - best: 0.1
Elastic Net: l1_ratio grid	{0.2, 0.5, 0.8} - best: 0.5

10 Results

Table 1 - Hyperparameter Tuning Summary (best CV R^2 during grid search):

Model	Search Method	Best Parameters	Best CV R^2
Ridge Regression	Grid Search	$\alpha = 100.0$	0.5714
Lasso Regression	Grid Search	$\alpha = 50.0$	0.5826
Elastic Net Regression	Grid Search	$\alpha = 0.1, l1_ratio = 0.5$	0.5835

Table 2 - 5-Fold Cross-Validation Performance (on training data):

Model	MAE	MSE	RMSE	R^2
Linear Regression	21566.95	1.0335×10^9	32148.61	0.5591
Ridge Regression	21586.35	1.0045×10^9	31693.08	0.5714
Lasso Regression	21545.73	9.7767×10^8	31267.66	0.5826
Elastic Net	21757.77	9.7546×10^8	31232.32	0.5835

Table 3 - Test Set Performance (held-out 15%):

Model	MAE	MSE	RMSE	R^2
Linear Regression	21624.06	9.9019×10^8	31467.32	0.5687
Ridge Regression	21632.30	9.8881×10^8	31445.39	0.5693
Lasso Regression	21619.62	9.8915×10^8	31450.79	0.5692
Elastic Net Regression	21825.28	9.9263×10^8	31506.00	0.5677

Table 4 - Coefficient Comparison (top 3 selected features):

Feature	Linear	Ridge	Lasso	Elastic Net
Income (USD)	47004.54	1119.45	0.00	117.28
Income Stability	791.16	772.93	700.50	618.03
Profession	-425.63	-431.13	-361.99	-455.76

On cross-validation, Elastic Net and Lasso clearly outperformed Ridge

and plain Linear Regression (R^2 around 0.58 versus 0.55–0.57). That gap almost disappears on the held-out test set, though - all four models land within about 0.002 of each other (R^2 0.5677–0.5693), with Ridge coming out marginally ahead this time. In other words, the differences between these models are small enough to be mostly noise once you move from cross-validation to a single held-out split; none of the four is meaningfully better than the others at this stage.

The coefficient table tells a more interesting story than the metrics do. Plain Linear Regression assigns Income (USD) a coefficient of 47,004.54 - wildly large compared to every other feature - which lines up with the duplicated Income/Property Age columns spotted in the EDA: multicollinearity like this is exactly what makes OLS coefficients unstable. Ridge pulls that same coefficient down to 1,119.45, Elastic Net down to 117.28, and Lasso removes it entirely (0.00), illustrating in a very direct way what each penalty is actually doing to the model.

11 Result Visualization

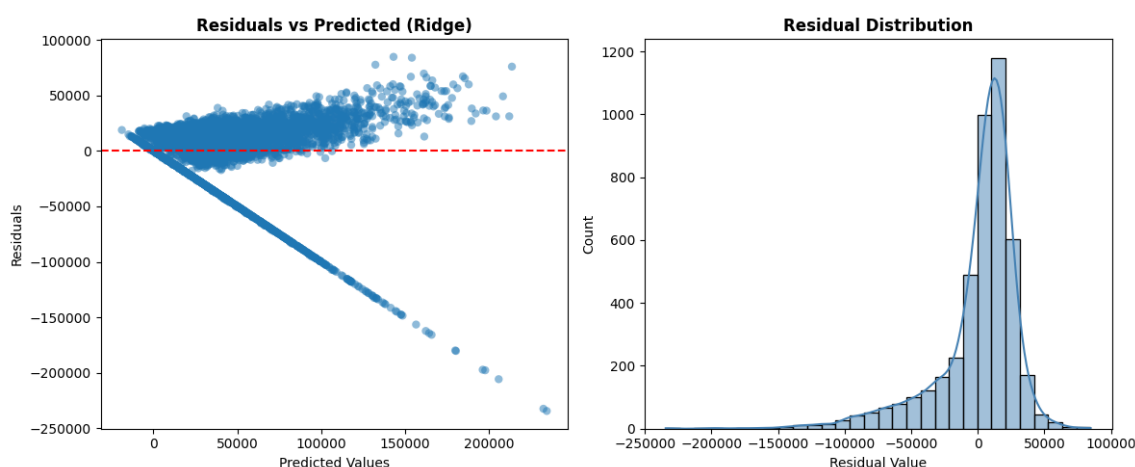


Figure 4: Residual analysis for the Ridge model - residuals vs. predicted values (left) and residual distribution (right)

Inference: Ridge was used here as the representative model for residual analysis. With test $R^2 \approx 0.57$, a substantial share of the variance in loan sanction amount is still unexplained, so a fair amount of residual spread on the left-hand scatter plot is expected rather than a sign of anything unusual. The right-hand histogram shows how those residuals are distributed around zero, which is the more useful of the two panels for checking whether the model’s errors are roughly symmetric or biased in

one direction - worth a closer look if the model is taken further, since any strong skew there would suggest the linear assumption is missing something systematic.

12 Discussion

The single most useful finding in this experiment isn't a metric at all - it's the coefficient table. All four models score within a fairly narrow band on the test set (R^2 0.568–0.569), so from a pure accuracy standpoint the choice between Linear, Ridge, Lasso and Elastic Net barely matters here. What does matter is stability: OLS's enormous, almost certainly spurious coefficient on Income (driven by its apparent duplication with Property Age) versus the much more sensible, shrunk values from Ridge, Lasso, and Elastic Net is a clean illustration of why regularization is worth using even when it doesn't move the headline metric much.

The gap between cross-validation and test performance is also worth a second look. Elastic Net and Lasso looked clearly better during tuning (R^2 around 0.58) but fell back in line with Ridge and Linear Regression on the test set. That's a reminder that a handful of CV folds can rank models slightly differently than a single held-out split does, especially when the underlying R^2 differences are this small to begin with.

There are a few clear limitations. The apparent Income/Property Age duplication was flagged but not corrected - if it's genuinely a data error, dropping one of the two columns and re-running everything would be a natural next step, and might change which features the ANOVA selection keeps. Label encoding was used for all categorical features, including ones with no natural order (Profession, Location), which imposes an artificial ordering that a tree-based model wouldn't need to worry about but a linear model does. Finally, all four models here are linear, so if the true relationship between applicant attributes and sanctioned amount is non-linear, none of them can fully capture it - a tree-based or boosted model would be a natural comparison point for a follow-up experiment.

13 Conclusion

Four linear regression variants were trained to predict loan sanction amount from applicant and property attributes, after cleaning, imputing, encoding, and selecting the top 10 features by ANOVA F-score. All four ended up

within a narrow band on test performance ($R^2 \approx 0.57$), so regularization strength didn't meaningfully change predictive accuracy in this case - but it did meaningfully change coefficient stability, most visibly by shrinking (Ridge, Elastic Net) or entirely removing (Lasso) an OLS coefficient that was almost certainly inflated by a data quality issue. That distinction - between "does it predict better" and "is it a more trustworthy model" - is the main takeaway to carry into a follow-up experiment, along with re-visiting the Income/Property Age columns and trying a non-linear model for comparison.